Modélisation mathématique, dénombrement théorique et recensement de séquences de jonglage

CHEYMOL William

Filière MP - Spécialité Informatique - Candidat No 32593

Sommaire

1. Introduction et problématique retenue
2. Modélisation mathématique : discrétisation d'une séquence de jonglage
3. Premier décompte : cartes de jonglage
4. Recensement complet : états et graphe d'états
5. Conclusion et ouvertures
6. Annexes

1 Introduction et problématique retenue

2 Modélisation mathématique : discrétisation d'une séquence de jonglage

3 Premier décompte : cartes de jonglage

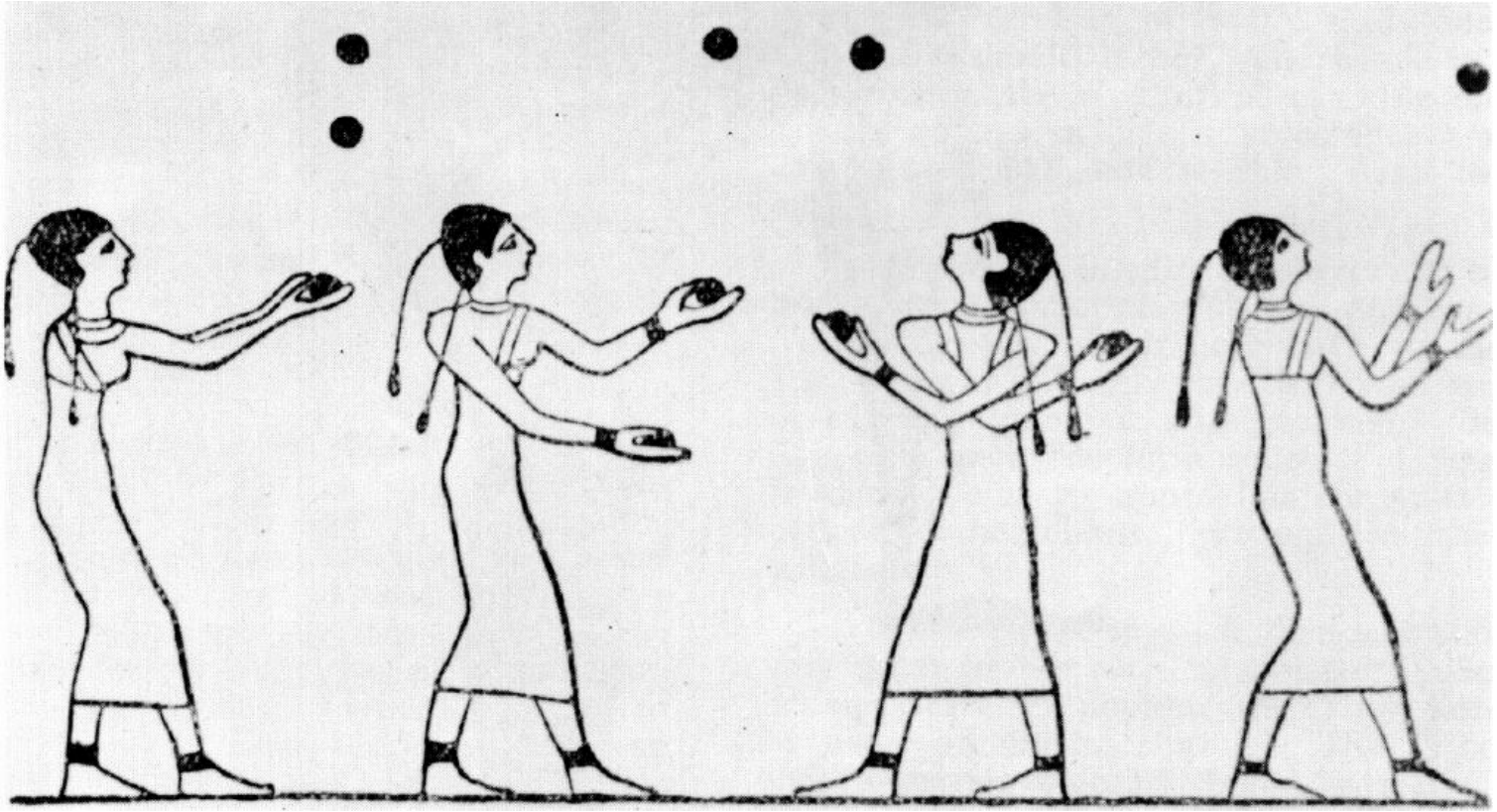
4 Recensement complet : états et graphe d'états

5 Conclusion et ouvertures

6 Annexes

Introduction

Fig. 1 - Peinture sur un mur égyptien (1994 - 1781 av. J-C) montrant des femmes en train de jongler avec des balles



Source : Wikipédia

Problématique retenue

Combien de séquences de jonglages existe–t–il et quels sont les outils mathématiques adéquats pour les compter et les recenser ?

1 Introduction et problématique retenue

2 Modélisation mathématique : discrétisation d'une séquence de jonglage

3 Premier décompte : cartes de jonglage

4 Recensement complet : états et graphe d'états

5 Conclusion et ouvertures

6 Annexes

Définitions et axiomes d'un jonglage simple

- Un exemple : *135* ou *135135135*...

Fig. 2 - Lancers pairs

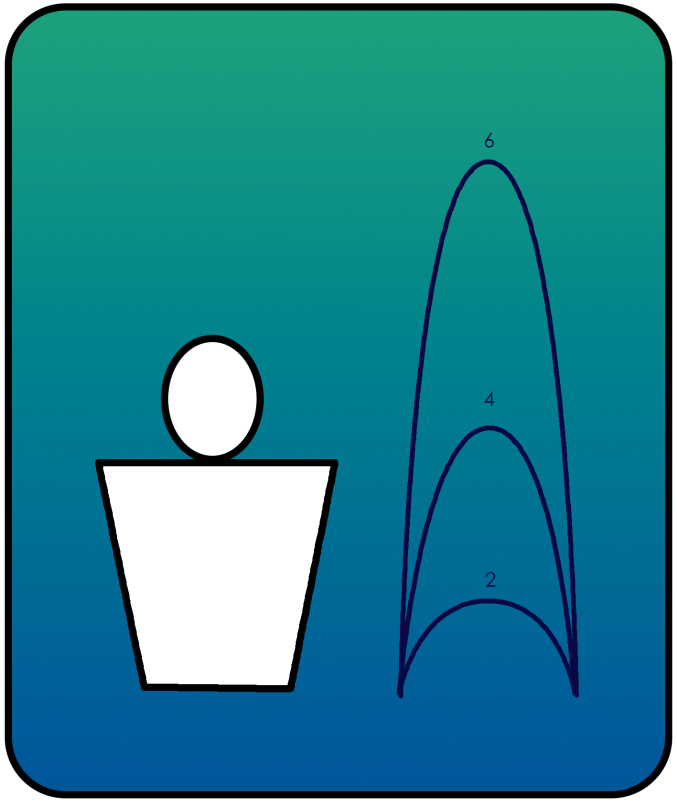
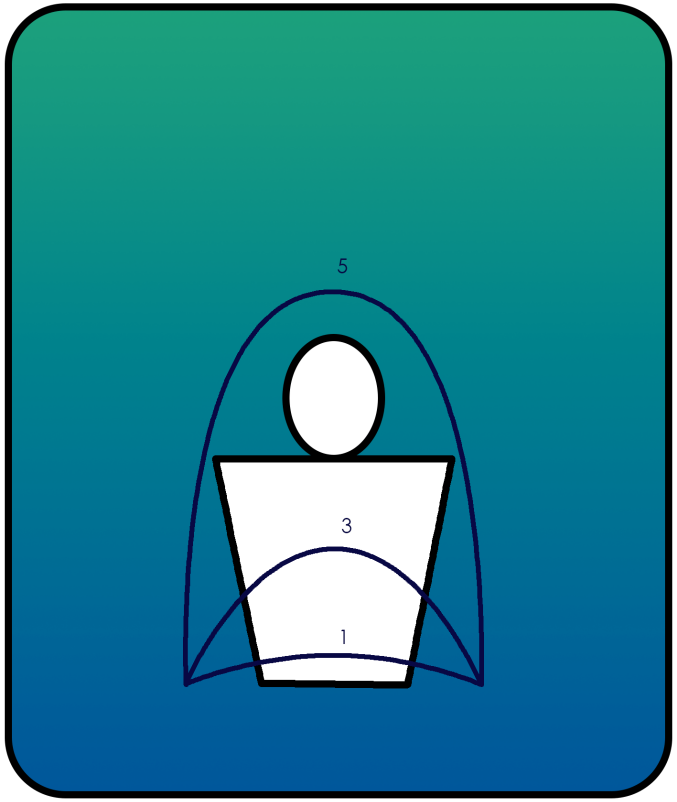


Fig. 3 - Lancers impairs



Paramètres et validité d'une séquence

Paramètres d'une séquence

Le nombre de balles b, hauteur maximale des lancers H et période de la séquence t sont les trois paramètres d'une séquence (en plus évidemment de l'ordre des lancers) .

Théorème de la moyenne

- Le nombre de balles b nécessaires pour jongler une séquence est égale à la moyenne des lancers la constituant.
- Dans le cas où ce nombre n'est pas entier, la séquence n'est pas valide.

1 Introduction et problématique retenue

2 Modélisation mathématique : discrétisation d'une séquence de jonglage

3 Premier décompte : cartes de jonglage

4 Recensement complet : états et graphe d'états

5 Conclusion et ouvertures

6 Annexes

Cartes de jonglage : présentation

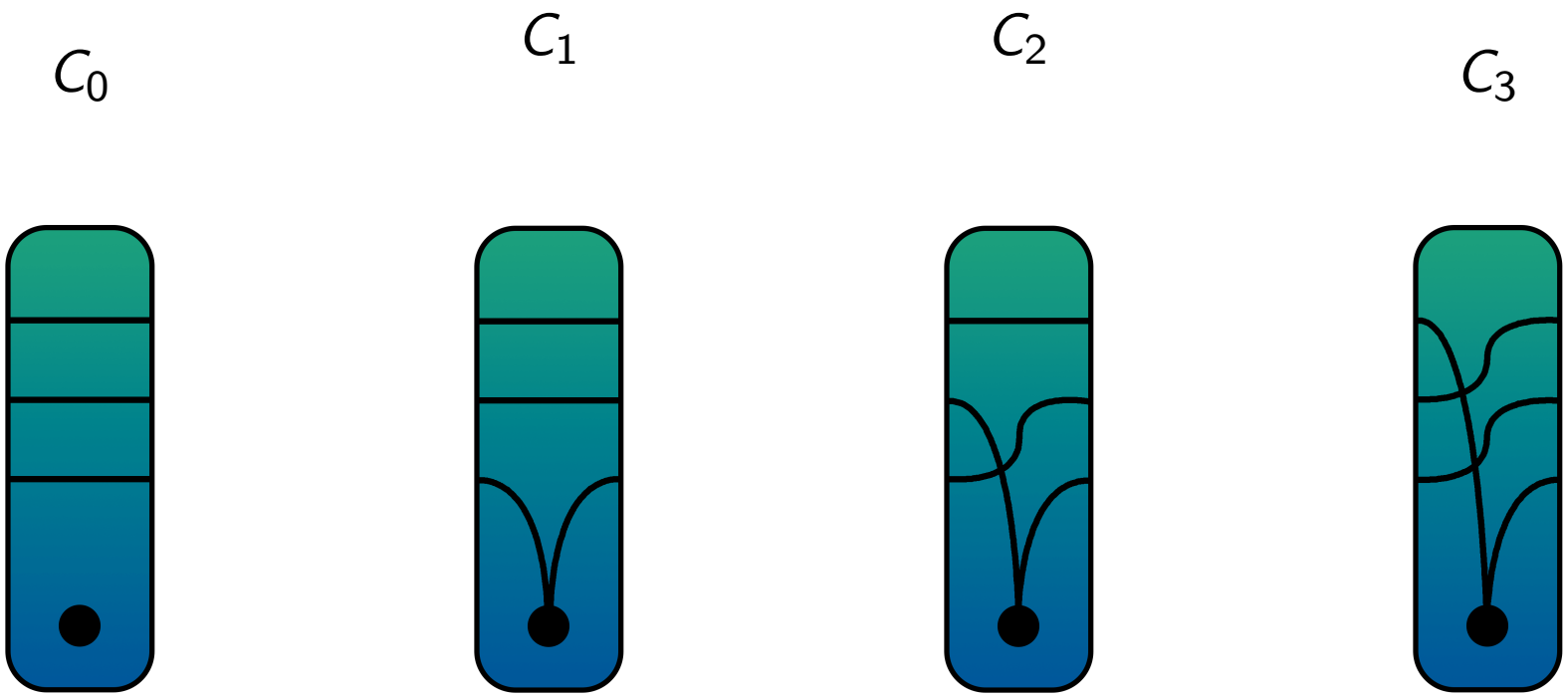


Fig. 4 - Les cartes de jonglage permettant de construire les séquences jusqu'à 3 balles

Créer des séquences grâce aux cartes de jonglage

- Un exemple : la séquence *4053*

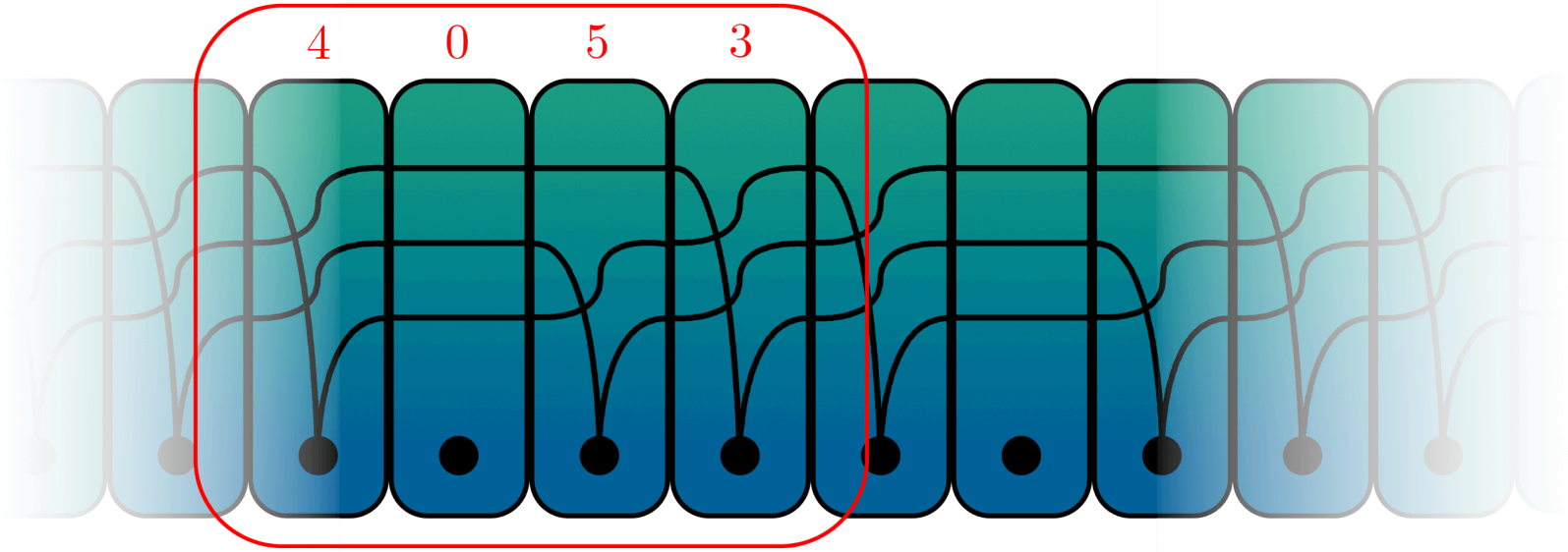


Fig. 5 - La séquence *4053* construite à partir de cartes de jonglage

Dénombrement avec les cartes de jonglage

Nombres de séquences jonglabes

- Le nombre de séquences jonglabes de période t et au plus à b balles vaut :

$$S^{\leq}(b,t) = (b+1)^t$$

- Le nombre de séquences jonglables de période t et à b balles vaut :

$$S(b,t) = S^{\leq}(b,t) - S^{\leq}(b-1,t) = (b+1)^t - b^t$$

- Le nombre de séquences jonglables pour une période maximale T et pour un nombre de balles maximal B vaut :

$$\boxed{S(B,T) = \sum_{b=1}^{B} \sum_{t=1}^{T} S(b,t)}$$

1 Introduction et problématique retenue

2 Modélisation mathématique : discrétisation d'une séquence de jonglage

3 Premier décompte : cartes de jonglage

4 Recensement complet : états et graphe d'états

5 Conclusion et ouvertures

6 Annexes

États

Un exemple

$$E_1 = \boxed{0110}$$

$$\boxed{0110} \xrightarrow[0]{} \boxed{1100} \xrightarrow[3]{} \boxed{1010} \xrightarrow[3]{} \boxed{0110}$$

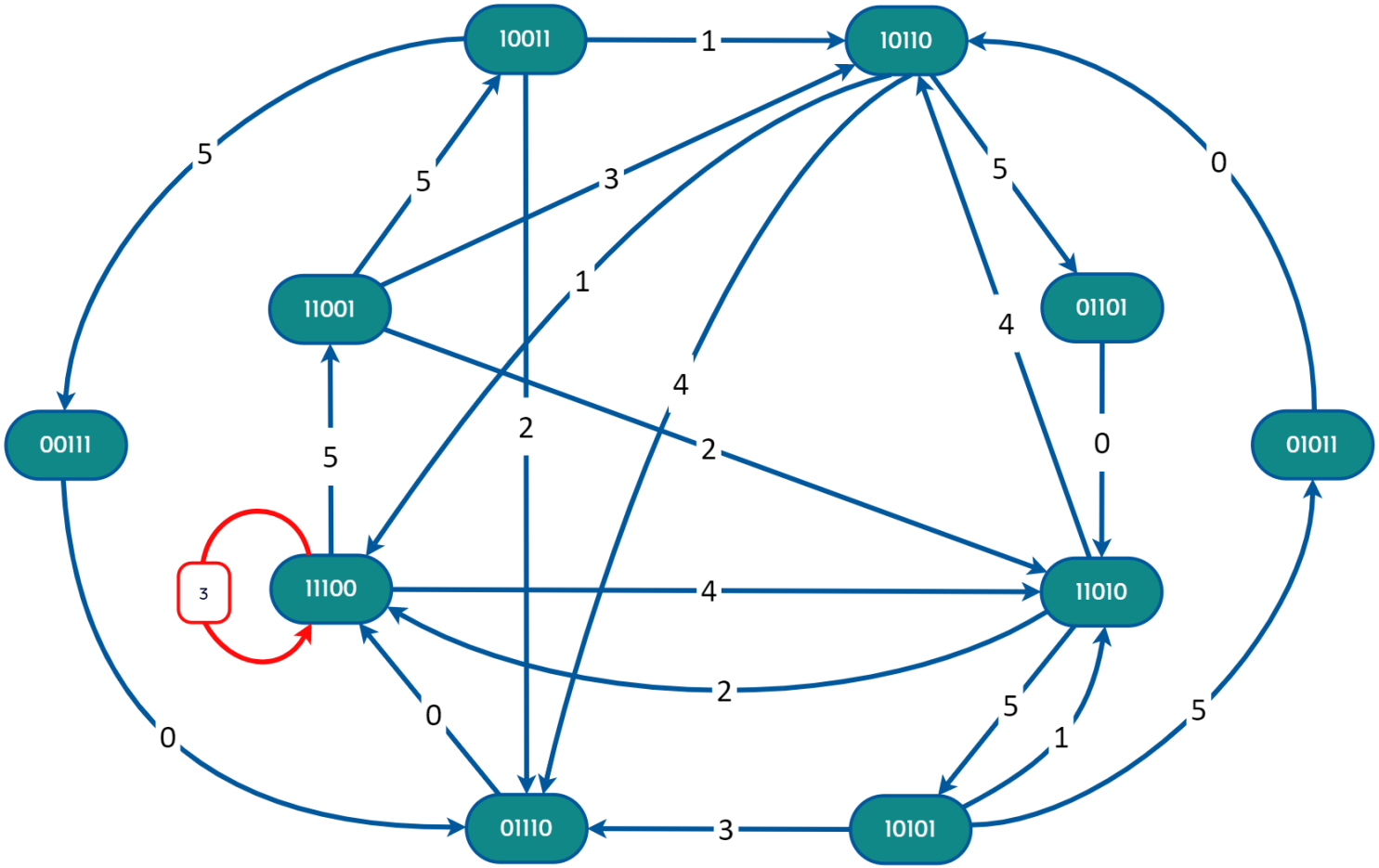
Ceci permet d'obtenir la séquence *033*.

Une propriété remarquable

Toutes les séquences trouvées de cette manière sont jonglables.

Graphe d'états

Fig. 6 - Le graphe d'états $G_{3,5}$



Matrice d'adjacence

Théorème des séquences incluses dans le graphe d'états

Le nombre de séquences jonglabes à b balles, à une hauteur maximale H et de période t correspond exactement au nombre de chemins fermés de longueur t dans le graphe d'états $G_{b,H}$.

Matrice d'adjacence du graphe $G_{3,5}$

La matrice d'adjacence correspondante au graphe précédent vaut :

$$M_{3,5,1} = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \\ G \\ H \\ I \\ J \end{array} \begin{array}{c} \begin{array}{cccccccccc} A & B & C & D & E & F & G & H & I & J \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix} \end{array}$$

Itérées d'une matrice et nombre de chemins

Théorème

Si A est une matrice d'adjacence d'un graphe fini (orienté ou non) , les nombres de parcours de longueur k entre deux sommets sont donnés par la matrice A^k.

Exemple : $b = 3$, $H = 5$ et $t = 2$

Le nombre de chemins fermés de longueur 2 vaut la somme des coefficients diagonaux de la matrice :

$$M_{3,5,2} = (M_{3,5,1})^2 = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \\ F \\ G \\ H \\ I \\ J \end{array} \begin{array}{c} \begin{array}{cccccccccc} A & B & C & D & E & F & G & H & I & J \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 2 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 2 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 2 & 2 \\ 1 & 0 & 1 & 2 & 0 & 1 & 2 & 0 & 0 & 2 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 & 1 & 2 & 2 \\ 0 & 0 & 0 & 0 & 1 & 1 & 2 & 1 & 2 & 2 \end{pmatrix} \end{array}$$

Trace et calcul final

Trace d'une matrice

Pour $A = \underset{1 \leqslant i,j \leqslant n}{(a_{ij})} \in \mathcal{M}_n(\mathbb{R})$, on définit sa trace par : $tr(A) = \sum_{i=1}^{n} a_{ii}$

Calcul final

Le nombre de séquences jonglables avec un nombre maximal de balles B, à une hauteur maximale H et de période maximale T vaut :

$$\boxed{S_{jonglabes}(B, T, H) = \sum_{b=1}^{B} \sum_{t=1}^{T} \frac{tr((M_{b,H})^t) - \gamma_{b,H,t}}{t}}$$

où $\gamma_{b,H,t} = \sum_{\substack{d|t \\ d \neq t}} S_{jonglabes}(B, d, H)$ représente le nombre de séquences qui sont des répétitions de séquences déjà rencontrées.

Calcul informatique et recensement

Première partie du programme

La première partie du programme conduit directement au résultat suivant, en éxécutant la fonction allpatternsam :

```
def all_patternsam():
    c=0
    for b in range(10):
        c+=num_patterns(adjacency_matrix(b,10),10)
    return c
```

```
>>> all_patternsam()
1342382
```

Deuxième partie du programme

Dans la deuxième partie du programme, on s'intéresse plus en détail au recensement des séquences et à obtenir par exemple une séquence aléatoire en fonction de nos préférences.

```
def random_pattern(b,h,n):   #Renvoie un pattern au hasard selon les paramètres entrés.
    F=census_filtered(b, h, n)
    x=random.randint(0, len(F)-1)
    return F(x)
```

```
>>> random_pattern(3,5,3)
'234'

>>> random_pattern(4,6,5)
'06662'
```

1 Introduction et problématique retenue

2 Modélisation mathématique : discrétisation d'une séquence de jonglage

3 Premier décompte : cartes de jonglage

4 Recensement complet : états et graphe d'états

5 Conclusion et ouvertures

6 Annexes

Conclusion et ouvertures

Conclusion

- Différents outils à notre disposition selon nos besoins
- Un paramétrage essentiel pour un décompte efficace

Poussée de l'étude

Étude menée ne prenant aucunement en compte l'énorme variété de styles de jonglages : simple, multiplex, etc. ou encore les différents types de lancers, les types de récupération des objets, ou encore la nature de ces objets.

Merci de votre écoute !

Source : Image extraite de sa vidéo YouTube
Catch me - Juggling by Taylor Glenn

Fig.7 - Taylor Glenn, jongleuse américaine effectuant un jonglage à quatre quilles

1 Introduction et problématique retenue

2 Modélisation mathématique : discrétisation d'une séquence de jonglage

3 Premier décompte : cartes de jonglage

4 Recensement complet : états et graphe d'états

5 Conclusion et ouvertures

6 Annexes

Annexe 1 : Code Python - partie 1

```python
import numpy as np
import numpy.linalg as np_linalg
import random
import time

#%% Préliminaires

def all_binary_lists(n):   #Génère toutes les listes de longueur n constituées de 0 et de 1.
    if n == 1:
        return [[0], [1]]
    else:
        R = all_binary_lists(n - 1)
        rep = []
        for e in R:
            rep.append(e + [0])
            rep.append(e + [1])
        return rep

def list_sum(L):   #Somme des éléments d'une liste
    if L == []:
        return 0
    elif len(L) == 1:
        return L[0]
    else:
        return L[0] + list_sum(L[1:])

#%% Partie permettant d'établir le calcul de la matrice d'adjacence mq

def all_states(b, h):   #Tous les états pour le nombre de balles b et la hauteur maximale h.
    rep = all_binary_lists(h)
    E = [e for e in rep]
    for L in rep:
        if list_sum(L) != b:
            E.remove(L)
    return E

def next_state(E, l):   #Génère l'état suivant à partir d'un état et d'un lancer.
    b = list_sum(E)
    R = [e for e in E[1:]] + [0]
    if l == 0:
        if list_sum(R) != b:
            return [], l
        else:
            return R, l
    elif R[l - 1] == 1:   #Si lancer impossible
        return [], l
    elif list_sum(R) == b:   #Si pas de balle disponible
        return [], l
    else:
        R[l - 1] = 1
        return R, l
```

Annexe 1 : Code Python - partie 2

```python
def all_next_states(E):   #Génère tous les états suivants possibles à partir d'un état.
    R = []
    L = []
    b = len(E)
    for k in range(b + 1):
            (N, l) = next_state(E, k)
            if N != []:
                R.append(N)
                s = "{l}".format(l=l)
                L.append(s)
    return R, L

def adjacency_matrix(b, h):   #Génère la matrice d'adjacence pour le nombre de balles b et la hauteur maximale
    M = []
    AS = all_states(b, h)
    for S in AS:
        L = [0 for n in range(len(AS))]
        AN, P = all_next_states(S)
        for p in AN:
            i = AS.index(p)
            L[i] = 1
        M.append(L)
    return np.array(M)

#%% Partie arithmétique concernant les diviseurs d'un entier quelconque

def divisors(k):  #Renvoie les diviseurs d'un entier quelconque.
    D = []
    c = int(k ** (1 / 2))
    for i in range(1, c + 1):
        if k % i == 0:
            D.append(i)
            if k // i not in D:
                D.append(k // i)
    D.remove(k)
    return D

#%% Partie concernant les calculs sur les matrices (carrées)

def matrix_trace(M):   #Renvoie la trace d'une matrice.
    t = 0
    for k in range(0, M.shape[0]):
        t = t + M[k, k]
    return t

def matrix_power(M, n):   #Renvoie la matrice à la puissance souhaitée.
    return np_linalg.matrix_power(M, n)
```

Annexe 1 : Code Python - partie 3

```python
#%% Partie concernant le dénombrement final à partir des matrices d'adjacence

def new_patterns(A, n):   #Renvoie les nouveaux patterns, sans ceux étant déjà des répétitions des autres.
    S = []
    for k in range(1, n + 1):
        t = matrix_trace(matrix_power(A, k))
        L = divisors(k)
        for h in L:
            t = t - S[h - 1]
        S.append(t)
    return S

def num_patterns(A, n):   #Renvoie le nombre final de patterns, sans les cycles.
    S = new_patterns(A, n)
    P = [int(S[k] / (k + 1)) for k in range(0, len(S))]
    return sum(P)

def all_patternsam():
    c=0
    for b in range(10):
        c+=num_patterns(adjacency_matrix(b,10),10)
    return c


#%% Partie sur la génération des séquences pour nos 3 paramètres, réduisant la complexité et renvoyant les patterns

# On dispose de allnexts, qui permet d'avoir tout les états possibles suivants pour un état donné, avec également les lancers correspondants à chaque état possible.
# On veut maintenant une fonction qui permet de renvoyer les séquences suivantes pour une liste d'état donnée avec les lancers.

def next_all_states(E, S):  #Fonction préliminaire qui renvoie toutes les états suivants pour une liste d'état ainsi que les séquences " complétées " avec les nouveaux lancers.
    EN = []
    SN = []
    for k in range(len(E)):
        AN1, AN2 = all_next_states(E[k])
        np_len = len(AN1)
        EN += AN1
        for j in range(np_len):
            SN.append(S[k] + AN2[j])
    return EN, SN

def all_patterns(E, n):    #Fonction qui renvoie ici les séquences de longueur exactement n partant d'un certain état.
    A1, A2 = all_next_states(E)
    for j in range(n - 1):
        X, Y = next_all_states(A1, A2)
        A1 = X
        A2 = Y
    P = []
    for k in range(len(A1)):
        if A1[k] == E:
            P.append(A2[k])
    return P
```

Annexe 1 : Code Python - partie 4

```python

def census(b, h, n):  #Généralisation supplémentaire en partant de tous les états pour nos 3 paramètres.
    C = []
    AS = all_states(b, h)
    for k in AS:
        C += all_patterns(k, n)
    return C

def census_filtered(b, h, n):
    C = census(b, h, n)
    encountered_patterns = set()  # Ensemble pour stocker les patterns rencontrés
    unique_patterns = []  # Liste pour stocker les patterns uniques

    for pattern in C:
        if pattern not in encountered_patterns and not contains_cycle(pattern,encountered_patterns):
            unique_patterns.append(pattern)
            encountered_patterns.add(pattern)

    return unique_patterns

def contains_cycle(pattern, patterns):
    for p in patterns:
        if is_equal(pattern, p):
            return True
    return False

def census_up_to_period(b, H, N):   #Même chose jusqu'à une certaine période avec un filtrage supplémentaire : enlèvement des répétitions. On suppose cette fois la hauteur H maximale, car le résultat contient tous les patterns pour une hauteur inférieure.
    CP = []
    start_time = time.perf_counter()
    for n in range(1, N + 1):
        F = census_filtered(b, H, n)
        CP = CP + [e for e in F if no_rep(e, CP)]
    end_time = time.perf_counter()
    execution_time = end_time - start_time
    print("Temps d'exécution :", execution_time, "secondes")
    return CP, len(CP)


#%% Partie concernant la manipulation des chaînes de caractères

def is_equal(u, v):  #Permet de savoir si la chaîne de caractères u est un cycle de v.
    return len(u) == len(v) and v in (u + u)

def is_repetition(u, v):   #Permet de savoir si la chaîne de caractères v est une répétition de u.
    if len(u) % (len(v)) == 0:
        return u == (len(u) // len(v)) * v

def no_rep(u, L):   #Permet de vérifier si une chaîne de caractères est une répétition d'une autre.
    for k in L:
        if is_repetition(u, k):
            return False
    return True
```

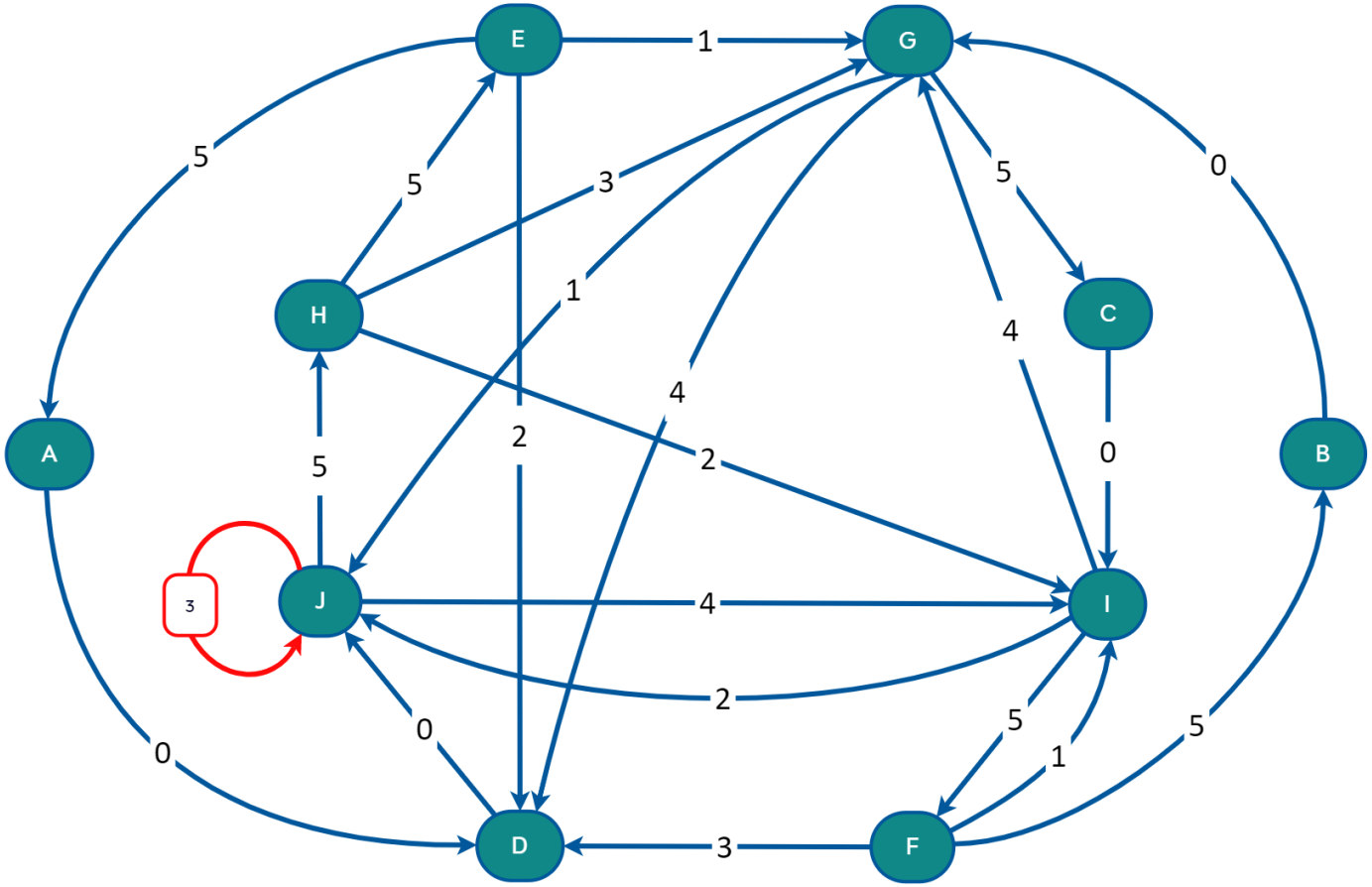
Annexe 1 : Code Python - partie 5

```python
#%% Partie bonus, concernant la génération de patterns aléatoirement

def random_pattern(b,h,n):   #Renvoie un pattern au hasard selon les paramètres entrés.
    F=census_filtered(b, h, n)
    x=random.randint(0, len(F)-1)
    return F[x]
```

Annexe 2 : Numérotation choisie pour la matrice $M_{3,5}$

Fig. 8 - La matrice $M_{3,5}$ avec sommets numérotés



Annexe 3 : Une autre expression possible de $S_{jonglabes}(B, T, H)$ - partie 1

La fonction γ

L'expression donnée est juste, et (rappel) on a $\gamma_{b,H,t} = \sum_{d|t} tr((M_{b,H})^d)$

Contournement du problème dans le programme Python

Le problème a été réglé dans le programme Python grâce à la fonction newpatterns :

```
def new_patterns(A, n):   #Renvoie les nouveaux patterns, sans ceux étant déjà des répétitions des autres.
    S = []
    for k in range(1, n + 1):
        t = matrix_trace(matrix_power(A, k))
        L = divisors(k)
        for h in L:
            t = t - S[h - 1]
        S.append(t)
    return S
```

Annexe 3 : Une autre expression possible de $S_{jonglabes}(B, T, H)$ - partie 2

Expression à l'aide de la fonction de Möbius

On peut aussi écrire, résultat découlant de l'utilisation des cartes de jonglage :

$$\boxed{S_{jonglabes}(B, T, H = B) = \sum_{b=1}^{B} \sum_{t=1}^{T} \frac{1}{t} \sum_{d|t} \mu\left(\frac{t}{d}\right) \left((b+1)^d - b^d\right)}$$

où μ représente la fonction de Möbius :

$$\forall n \in \mathbb{N},\ \mu(n) = \begin{cases} 1 \text{ si } n = 1 \text{ ou } n \text{ possède un nombre pair de facteurs premiers distincts} \\ -1 \text{ si } n \text{ possède un nombre impair de facteurs premiers distincts} \\ 0 \text{ si } n \text{ possède au moins un facteur premier répété} \end{cases}$$

Annexe 4 : La difficulté exponentielle du jonglage

Le niveau usuel d'un jongleur professionnel

- La plupart des jongleurs d'un niveau confirmé peinent à acquérir la cascade à 7 balles.
- Les jongleurs professionnels n'arrivent pas à acquérir une cascade à 9 balles. Il est difficile pour eux de faire des variations d'une cascade à 7 balles.

Quelques records historiques

- En 2017, Alex Barron arrive à éxécuter un flash à 14 balles (14 lancers)
- Enrico Rastelli (1896 - 1931) , considéré comme le meilleur jongleur de tous les temps savait jongler à 10 balles, à 8 bâtons ou encore avec 8 assiettes. Il s'entraînait 10 heures par jour.

Annexe 5 : Ressources bibliographiques

[1] BURKARD POLSTER : The Mathematics of Juggling : *Édition Springer, 2003, ISBN : 0-387-95513-5*
[2] APMEP : *Mathématiques du Jonglage*
[3] TRUZZI : *Notes Towards a History of Juggling*
[4] JACK BOYCE : *Juggling Lab*
[5] FLORENT HIVERT : *Jonglerie, automate et combinatoire : 30 minutes pour comprendre, 13 novembre 2006, Université de Rouen*